1. What is the difference between method overloading and method overriding in terms of inheritance?

- Method Overloading:
  - Occurs within the same class (inheritance is not mandatory).
  - Methods have the same name but different parameter lists (different number or type of arguments).
  - Resolved at compile-time (static polymorphism).
- Method Overriding:
  - Occurs across parent-child classes (inheritance is mandatory).
  - Child class provides a new implementation of a method that already exists in the parent class with the same signature (name, parameters, return type).
  - Resolved at runtime (dynamic polymorphism).

---

2. Can you inherit a private method? If not, why?

- No, private methods cannot be inherited.
- Reason: private methods are only accessible within the class in which they are defined.
- Child classes don't have visibility of the parent's private methods.
- However, you can define a new method with the same name in the child class, but it won't be overriding—it's just a new method.

---

3. Can a final class be inherited? Give an example.

- No, a final class cannot be inherited.
- Reason: final keyword prevents extension to ensure immutability, security, or design constraints.

Example:

```
final class Parent {
   void show() {
      System.out.println("This is a final class");
   }
}

class Child extends Parent {   //  Compilation error
   // Cannot inherit from final Parent
}
```

---

4. What will happen if a parent class reference points to a child class object?

- This is called upcasting.
- You can call methods that are defined in the parent class, but if they are overridden in the child class, the child's version will be executed (runtime polymorphism).

Example:

```
class Parent {
   void show() { System.out.println("Parent method"); }
}

class Child extends Parent {
   void show() { System.out.println("Child method"); }
}

public class Test {
   public static void main(String[] args) {
      Parent obj = new Child();  // Parent reference, Child object
      obj.show(); // Output: Child method
   }
}
```

---

5. Can you access the parent class constructor from a child class? If yes, how?

- Yes, by using the super() keyword.
- super() must be the first statement inside the child class constructor.

Example:

```
class Parent {
   Parent() {
      System.out.println("Parent constructor");
   }
}

class Child extends Parent {
   Child() {
      super(); // Calls Parent constructor
      System.out.println("Child constructor");
   }
}

public class Test {
   public static void main(String[] args) {
      Child obj = new Child();
   }
}
```

Output:

```
Parent constructor
Child constructor
```